DATA ENGINEERING

Schema Evolution Visual Guide

Change your data shape without breaking producers or consumers.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

The Problem: Schemas Change

Add a field, rename a column — and every consumer breaks at 2 AM.

Without a compatibility contract, every schema change is a coordinated deployment across teams — or an outage.

STEP 2/6

Compatibility Modes

Avro and Protobuf define exactly which changes are safe.

BACKWARD

New readers read old data. Add optional fields freely.

FORWARD

Old readers read new data. Only remove optional fields.

FULL

Both directions. Safest, most restrictive.

NONE

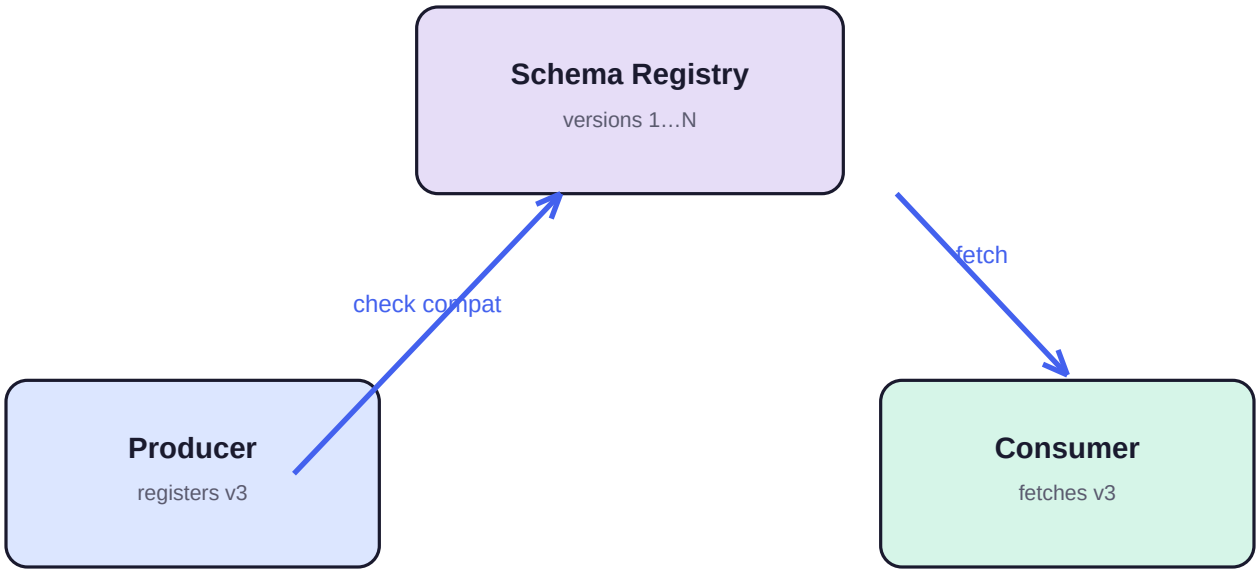
No guarantees. Chaos mode — never in production.

Default to BACKWARD: consumers upgrade first, then producers start writing new fields. Old consumers ignore what they don't know.

STEP 3/6

Schema Registry: The Gatekeeper

A central service that versions schemas and enforces compatibility.



Producer registers the new schema; the registry REJECTS it if it breaks compatibility with the previous version. Bad schemas never reach the wire.

- Wire format carries only a schema ID — tiny payloads
- Consumers cache schemas locally, fetch on unknown ID
- One registry per organization, not per team

STEP 4/6

Safe vs Unsafe Changes

The cheat sheet for evolving Avro/Protobuf schemas.

SAFE ✓

- Add optional field (with default)
- Remove optional field
- Widen: int → long

UNSAFE ✗

- Rename a field (it's add+remove!)
- Change type: string → int
- Remove a required field

Renaming is the classic trap: to consumers it's a deletion plus an addition. Deprecate the old field, add the new one, remove later.

STEP 5/6

The Safe Migration Recipe

Expand, migrate, contract — never a big bang.

Each phase is independently deployable and rollback-safe. The old field stays until every consumer has migrated — measure before contracting.

- Phase 1: producers write both fields; consumers read old
- Phase 2: consumers switch to new; keep dual-write
- Phase 3: stop writing old; registry retires it

STEP 6/6

Avro vs Protobuf

Two ecosystems, same discipline.

Avro

schema travels with data

- JSON schemas, dynamic
- Hadoop/Spark/Kafka native
- Reader/writer schema resolution

Protobuf

code-generated stubs

- IDL + generated classes
- gRPC ecosystem standard
- Field numbers = identity

Protobuf field numbers are forever — never reuse a retired number. Avro resolves by name — never rename casually. Both rules prevent silent corruption.

STEP ✓/✓

Key Takeaways

Schema Evolution — Remember

- Compatibility modes (backward/forward/full) define exactly which schema changes are safe.
- A schema registry versions schemas and rejects incompatible ones before they reach the wire.
- Safe: add optional fields with defaults. Unsafe: renames and type changes.
- Migrate with expand → migrate → contract; never big-bang a schema change.
- Protobuf: never reuse field numbers. Avro: fields are resolved by name.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev